Main Board Design

Circuit Analyses and Component Selection

Raspberry Pi Zero W & Zero 2 W

Adafruit TFT 2.8in Touchscreen Display
TSC2007 $\cdot SPI and I2C$

LED boards $\cdot Testing \cdot Connectors \cdot Power$

April 25, 2026

This document covers design ideas, component selection, and sub-circuit diagrams.
It is written for a technically-minded reader who may not have prior electronics design experience.

Contents

1 Magnetic Buzzer **2**
1.1 Overview . . . 2
1.2 Circuit Description . . . 2
1.3 Parts List . . . 3
1.4 Programming Notes . . . 3
1.5 Example Code . . . 3

2 Extending MOSFETs with Shift Register **4**
2.1 Overview . . . 4
2.2 Circuit Description . . . 5
2.3 Circuit Diagram . . . 5
2.4 Parts List . . . 6
2.5 Programming Notes . . . 6
2.6 Example Code . . . 6

3 High-Side Control of 5 V Load with MOSFET **7**
3.1 Overview . . . 7
3.2 The Level-Shifting Problem . . . 8
3.3 Circuit Description . . . 8
3.4 Circuit Diagram . . . 8
3.5 Parts List . . . 9
3.6 Logic Table . . . 9
3.7 Example Code . . . 9

4 Rotary Encoders **10**
4.1 Overview . . . 10
4.2 How a Rotary Encoder Works . . . 10
4.3 Contact Bounce and Debouncing . . . 11
4.4 Wiring . . . 11
4.5 Pin Restrictions on the Raspberry Pi Zero . . . 12
4.6 Setting Up Interrupts in Python . . . 12
4.7 Two Encoders with Multiple Interrupts . . . 13
4.8 Temporarily Reassigning an Encoder . . . 14
4.9 Adafruit 1651 Touchscreen Interrupt (TSC2007) . . . 15
4.10 Parts List . . . 15

1 Magnetic Buzzer

1.1 Overview

The alert sounder is a passive magnetic buzzer driven by a pulse-width modulated (PWM) signal from the Raspberry Pi Zero. A passive magnetic buzzer is preferred over an active buzzer or piezoelectric transducer because its tone is warm and rounded rather than harsh, and because both pitch and volume are controllable in software via the PWM frequency and duty cycle respectively.

The buzzer operates from the 5 V supply rail and draws approximately 30 mA to 40 mA at full drive. Since this exceeds the safe output current of a GPIO pin, and because the supply voltage exceeds the 3.3 V GPIO logic level, a small-signal NPN bipolar junction transistor is used as a low-side switch between the buzzer and ground.

1.2 Circuit Description

The GPIO pin drives the base of the NPN transistor through a current-limiting resistor R1. When the GPIO is high, the transistor saturates and connects the buzzer's negative terminal to ground, completing the circuit through the 5 V supply. When the GPIO is low, the transistor is cut off and the buzzer is de-energised.

Because the buzzer is an inductive load, a flyback diode is placed across its terminals to suppress the reverse voltage spike that occurs when the transistor switches off. Without this diode the spike could damage the transistor.

Volume is controlled by adjusting the PWM duty cycle. A duty cycle of approximately 50% produces full volume; reducing the duty cycle lowers the average power delivered to the buzzer and therefore reduces loudness. Pitch is controlled by adjusting the PWM frequency. The range of 2000 Hz to 2500 Hz produces a clear, pleasant tone that is audible in both laboratory and outdoor environments. The buzzer is rated at approximately 85 dB to 90 dB at 10 cm.

The complete circuit is shown below.

```
RPi GPIO (PWM) ---[ R1 1k ]---+--- MMBT3904 Base
                              |
                        MMBT3904 Emitter --- GND

+5V ---[ Buzzer CEM-1203(42) ]---+--- MMBT3904 Collector
                                 |
                          1N4148W (flyback)
                          Anode  -> Collector side
                          Cathode -> +5V side
```

Reference	Part Number	Description	Package
BZ1	CEM-1203(42)	Passive magnetic buzzer, 5 V, 2048 Hz rated	12 mm through-hole
D1	1N4148W	Fast switching diode, flyback protection	SOD-123
Q1	MMBT3904	NPN small-signal BJT, 40 V, 200 mA	SOT-23
R1	R1K-0805	Resistor 1 kΩ, base current limiter	0805

Table 1: Magnetic buzzer circuit components

1.3 Parts List

1.4 Programming Notes

The buzzer is controlled using the RPi.GPIO software PWM facility. Any GPIO pin may be used, but a hardware PWM pin (GPIO 18 or GPIO 12 on the Raspberry Pi Zero) is preferred because software PWM can exhibit jitter under system load, which may cause audible roughness in the tone.

Key relationships are:

- **Frequency** sets pitch. The range 1500 Hz to 3000 Hz is recommended. Values below 1000 Hz may be too quiet for some magnetic buzzers, and values above 4000 Hz become increasingly harsh.
- **Duty cycle** sets volume. A value of 50 (50%) gives maximum loudness. Values below approximately 10 may be too quiet to be useful. Values above 50 do not increase loudness and may introduce distortion.
- **Duration** is implemented as a blocking `time.sleep()` call. Interruption logic is not implemented in this version.

The GPIO pin must be initialised as an output before the PWM object is created. The PWM object should be started with a duty cycle of zero and frequency set before each call to avoid a brief click at startup.

1.5 Example Code

```
import RPi.GPIO as GPIO
import time

BUZZER_PIN = 18  # Hardware PWM pin recommended

GPIO.setmode(GPIO.BCM)
GPIO.setup(BUZZER_PIN, GPIO.OUT)

_pwm = GPIO.PWM(BUZZER_PIN, 2000)
_pwm.start(0)
```

```
def sound_alert(frequency: int = 2000,
                volume: int = 50,
                duration: float = 1.0) -> None:
    """
    Sound the magnetic buzzer for a fixed duration.

    Parameters
    ----------
    frequency : int
        Tone frequency in Hz. Recommended range 1500-3000.
        Default 2000 Hz.
    volume : int
        Duty cycle as a percentage, 0-50. Values above 50
        do not increase loudness. Default 50 (full volume).
    duration : float
        Duration in seconds to sound the alert. Blocking.
        Default 1.0 second.
    """
    volume = max(0, min(50, volume))       # clamp to 0-50
    frequency = max(100, min(10000, frequency))  # clamp to safe range

    _pwm.ChangeFrequency(frequency)
    _pwm.ChangeDutyCycle(volume)
    time.sleep(duration)
    _pwm.ChangeDutyCycle(0)                # silence buzzer, leave PWM running
```

To sound a two-second alert at 2500 Hz at three-quarter volume:

```
sound_alert(frequency=2500, volume=37, duration=2.0)
```

To clean up GPIO resources on program exit:

```
_pwm.stop()
GPIO.cleanup()
```

2 Extending MOSFETs with Shift Register

2.1 Overview

The Raspberry Pi Zero provides a limited number of GPIO pins. Where multiple independent loads must be switched, a serial-to-parallel shift register allows eight MOSFET gates to be controlled from three GPIO pins. The device chosen is the **74HC595**, an 8-bit serial-in, parallel-out shift register that operates from 3.3 V and is widely available in an SMD package.

Data is clocked into the shift register serially and latched to the eight parallel outputs simultaneously, ensuring that all outputs update at the same instant. Although the 74HC595 can be driven by bit-banging three GPIO pins, the implementation here uses the hardware

SPI1 peripheral on the Raspberry Pi Zero for reliable timing and minimal CPU overhead. Only one MOSFET is energised at a time; the controlling byte therefore always has exactly one bit set.

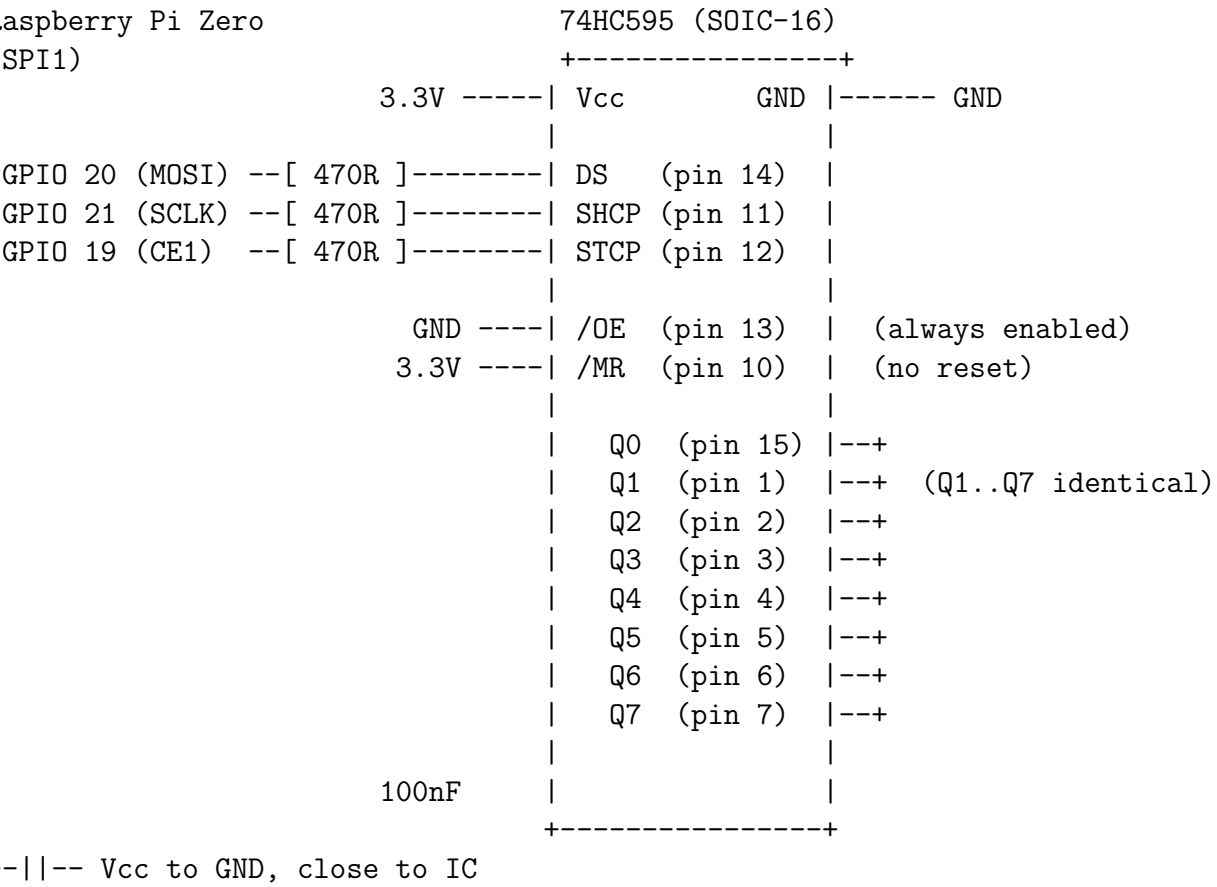
2.2 Circuit Description

The Raspberry Pi Zero SPI1 peripheral provides three signals: MOSI (data), SCLK (clock), and CE (chip enable, used here as the latch signal). Each signal passes through a 470 Ω series resistor before reaching the 74HC595 to protect the GPIO pins from transient currents.

The 74HC595 output enable pin (OE) is tied permanently to ground so that the parallel outputs are always active. The master reset pin (MR) is tied permanently to 3.3 V to prevent accidental clearing of the register. A 100 nF bypass capacitor is placed directly across the Vcc and GND pins of the IC.

Each of the eight parallel outputs drives a MOSFET gate through a 500 Ω series resistor. A 10 kΩ resistor from gate to ground ensures the gate is pulled firmly low when the corresponding output is de-asserted, preventing the MOSFET from floating into an undefined state.

2.3 Circuit Diagram



```
One MOSFET channel (repeated x8):

74HC595 Qn ---[ 500R ]---+--- MOSFET Gate (AO3400A or similar)
                         |
                        10k
                         |
                        GND                MOSFET Source --- GND
                                           MOSFET Drain  --- Load (-)
```

2.4 Parts List

Reference	Part Number	Description	Package
U1	74HC595	8-bit shift register, serial-in parallel-out	SOIC-16
Q1–Q8	AO3400A	N-channel MOSFET, 30 V, 5.7 A	SOT-23
R1–R3	R470-0603	470 Ω GPIO series resistors	0603
R4–R11	R500-0603	500 Ω MOSFET gate resistors	0603
R12–R19	R10K-0603	10 kΩ gate pull-down resistors	0603
C1	100nF ceramic	Bypass capacitor, Vcc to GND	0603

Table 2: Shift register MOSFET controller components

2.5 Programming Notes

The SPI1 peripheral on the Raspberry Pi Zero is accessed via the `spidev` library. SPI mode 0 (CPOL=0, CPHA=0) is compatible with the 74HC595. A clock speed of 500 kHz is more than adequate; the 74HC595 is rated to several megahertz but there is no benefit to running faster for low-speed MOSFET switching.

A single byte is transferred per operation. The byte has exactly one bit set to select one MOSFET channel, or is zero to de-energise all channels. The `xfer2()` call clocks out the data and automatically toggles the chip enable line, which is connected to the latch (STCP) pin, causing the outputs to update at the end of the transfer.

Note that SPI1 on the Raspberry Pi Zero requires the overlay `spi1-1cs` to be enabled in `/boot/config.txt`:

```
dtoverlay=spi1-1cs
```

2.6 Example Code

```
import spidev

SPI_BUS = 1       # SPI1
SPI_DEVICE = 0    # CE0

spi = spidev.SpiDev()
spi.open(SPI_BUS, SPI_DEVICE)
```

```
spi.max_speed_hz = 500000
spi.mode = 0


def set_mosfet(channel: int) -> None:
    """
    Energise a single MOSFET by channel number.

    Parameters
    ----------
    channel : int
        Channel to activate, 0 through 7.
        Pass -1 or any out-of-range value to de-energise all channels.
    """
    if 0 <= channel <= 7:
        byte = 1 << channel
    else:
        byte = 0
    spi.xfer2([byte])


def all_off() -> None:
    """De-energise all MOSFET channels."""
    spi.xfer2([0x00])
```

To activate channel 4 then turn everything off:

```
set_mosfet(4)
time.sleep(1.0)
all_off()
```

3 High-Side Control of 5 V Load with MOSFET

3.1 Overview

In circuits requiring both high-side and low-side switching, the load is interrupted at both the positive supply terminal and the ground return. The low-side switch is an N-channel MOSFET (AO3400A), which is straightforward to drive from a 3.3 V GPIO. The high-side switch requires a P-channel MOSFET because its source terminal connects to the positive supply rail.

The P-channel MOSFET chosen is the **AO3401A**, the complementary P-channel counterpart to the AO3400A. Both devices share the SOT-23 package and are produced by the same manufacturer, making them a well-matched pair. The AO3401A is rated at −30 V Vds and −4 A continuous drain current, well in excess of the anticipated load current.

3.2 The Level-Shifting Problem

A P-channel MOSFET turns on when its gate is pulled sufficiently below its source voltage. With the source at 5 V, the gate must be pulled close to ground to turn the device on, and must be pulled to 5 V to turn it off. A 3.3 V GPIO cannot swing to 5 V and therefore cannot turn the device fully off. A small-signal NPN transistor is used to solve this: it pulls the gate to ground when the GPIO is high (turning the P-channel on), and allows the gate to be pulled to 5 V through a resistor when the GPIO is low (turning the P-channel off).

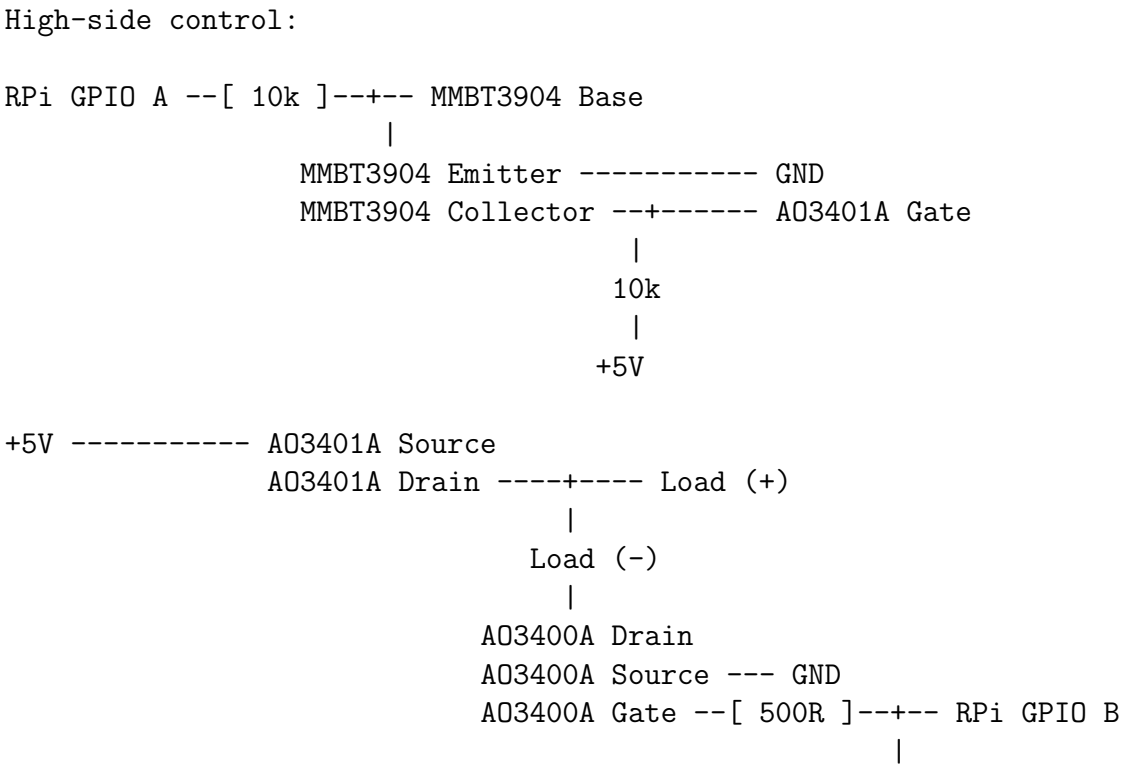
3.3 Circuit Description

The GPIO drives the base of an MMBT3904 NPN transistor through a 10 kΩ base resistor. The collector of the NPN connects to the gate of the AO3401A. A 10 kΩ resistor from the gate to 5 V pulls the gate high when the NPN is off, ensuring the P-channel is firmly off. When the NPN saturates, it pulls the gate to ground, turning the P-channel on.

The logic is inverted by the NPN stage: GPIO high turns the high-side MOSFET on, and GPIO low turns it off. This matches the behaviour of the low-side N-channel switch, so both MOSFETs are enabled by a high logic level on their respective GPIO pins. The load is energised only when both GPIO pins are high simultaneously.

When the supply voltage is 5 V and the gate is pulled to ground by the NPN, Vgs of the AO3401A is $0\,\text{V} - 5\,\text{V} = -5\,\text{V}$. The AO3401A gate threshold is $-1\,\text{V}$, so the device is driven well into conduction.

3.4 Circuit Diagram



```
                                                    10k
                                                     |
                                                    GND

  Load energised when: GPIO A = HIGH and GPIO B = HIGH
```

3.5 Parts List

Reference	Part Number	Description	Package
Q1	AO3401A	P-channel MOSFET, −30 V, −4 A, high-side switch	SOT-23
Q2	AO3400A	N-channel MOSFET, 30 V, 5.7 A, low-side switch	SOT-23
Q3	MMBT3904	NPN BJT, 40 V, 200 mA, gate driver	SOT-23
R1	R10K-0603	10 kΩ NPN base resistor	0603
R2	R10K-0603	10 kΩ P-channel gate pull-up to 5 V	0603
R3	R500-0603	500 Ω N-channel gate series resistor	0603
R4	R10K-0603	10 kΩ N-channel gate pull-down to GND	0603

Table 3: High-side and low-side MOSFET switch components

3.6 Logic Table

GPIO A (high-side)	NPN	AO3401A (P-channel)	GPIO B (AO3400A) (low-side)	Load
LOW	OFF	OFF	LOW	Off
LOW	OFF	OFF	HIGH	Off
HIGH	ON	ON	LOW	Off
HIGH	ON	ON	HIGH	**On**

Table 4: High-side and low-side switch logic

3.7 Example Code

```
import RPi.GPIO as GPIO

HIGH_SIDE_PIN = 23   # controls AO3401A via MMBT3904
LOW_SIDE_PIN  = 24   # controls AO3400A directly

GPIO.setmode(GPIO.BCM)
GPIO.setup(HIGH_SIDE_PIN, GPIO.OUT, initial=GPIO.LOW)
GPIO.setup(LOW_SIDE_PIN,  GPIO.OUT, initial=GPIO.LOW)


def load_on() -> None:
    """Energise the load by enabling both high-side and low-side switches."""
```

```
    GPIO.output(HIGH_SIDE_PIN, GPIO.HIGH)
    GPIO.output(LOW_SIDE_PIN,  GPIO.HIGH)


def load_off() -> None:
    """De-energise the load. Low side is opened first."""
    GPIO.output(LOW_SIDE_PIN,  GPIO.LOW)
    GPIO.output(HIGH_SIDE_PIN, GPIO.LOW)
```

The low-side switch is opened first in `load_off()` to interrupt current flow before the high side is released. This order avoids a brief condition where the high side is off but the low side is still on, which would momentarily short the load's negative terminal through the N-channel with no current path, producing no harm but making the sequence logically cleaner.

4 Rotary Encoders

4.1 Overview

Two rotary encoders are used in this design. One controls the alert buzzer volume and the other controls the display brightness. Either encoder may be temporarily reassigned by the application script to set other parameters as needed. The encoders do not include push-button switches.

A rotary encoder is a digital position sensor. Unlike a potentiometer, it has no end stops and produces no analog output. Instead, it produces a series of digital pulses as the shaft is rotated, with the direction of rotation encoded in the phase relationship between two output signals. Because the encoder has no absolute position, the application maintains a software counter that is incremented or decremented on each detent step.

4.2 How a Rotary Encoder Works

Inside the encoder are two switches, conventionally labelled A and B (sometimes CLK and DT). The switches are arranged so that they open and close in a specific sequence depending on the direction of rotation. As the shaft turns clockwise, A transitions before B. As the shaft turns counter-clockwise, B transitions before A. The relationship is a two-bit Gray code sequence:

Step	A	B
0	1	1
1	0	1
2	0	0
3	1	0

Table 5: Gray code sequence for clockwise rotation. Counter-clockwise reverses the order.

The simplest detection strategy is to trigger an interrupt on the falling edge of signal A, then immediately read the state of signal B. If B is high at that moment, rotation is clockwise; if B

is low, rotation is counter-clockwise. This single-edge approach is sufficient for a hand-turned control knob where speed is low.

4.3 Contact Bounce and Debouncing

The switches inside a mechanical rotary encoder bounce as they make and break contact, producing multiple spurious transitions per detent step. Without debouncing, the interrupt handler fires several times per click and the counter accumulates incorrect values.

Two debouncing strategies are available:

- **Hardware debouncing:** A small capacitor (typically 10 nF to 100 nF) from each encoder pin to ground, combined with a pull-up resistor (10 kΩ), forms an RC low-pass filter that smooths the bounce transients. This is the preferred approach for a production design as it requires no timing logic in software.
- **Software debouncing:** Record the timestamp of each interrupt and ignore subsequent triggers that arrive within a debounce window (typically 5 ms to 50 ms). The RPi.GPIO `add_event_detect()` call includes a built-in `bouncetime` parameter for exactly this purpose.

Both approaches may be combined. Hardware debouncing is recommended with a 100 nF ceramic capacitor on each encoder pin.

4.4 Wiring

Each encoder has two signal pins (A and B), a common pin, and optionally a supply pin. The common is connected to ground. Each signal pin is pulled up to 3.3 V through a 10 kΩ resistor (the Raspberry Pi internal pull-up may be used instead, activated in software). A 100 nF ceramic capacitor from each signal pin to ground provides hardware debouncing.

```
3.3V ---[ 10k ]---+--- Encoder A pin
                  |
                100nF
                  |
                 GND

3.3V ---[ 10k ]---+--- Encoder B pin
                  |
                100nF
                  |
                 GND

Encoder Common --- GND
```

4.5 Pin Restrictions on the Raspberry Pi Zero

All GPIO pins on the Raspberry Pi Zero support interrupts via the RPi.GPIO library. There is no hardware restriction limiting interrupts to specific pins, unlike some microcontrollers. The Linux kernel `gpio` subsystem services all GPIO interrupts through a unified interface.

Practical limits to be aware of:

- **Timing jitter:** Because the Raspberry Pi runs Linux, interrupt latency is not deterministic. Typical latency is 1 ms to 5 ms under light load, but may spike under heavy CPU or I/O activity. For a hand-turned knob this is entirely acceptable; for high-speed pulse counting it would not be.
- **Number of interrupts:** There is no fixed limit on the number of simultaneous GPIO interrupts. Each `add_event_detect()` call starts a kernel-level watch on that pin. In practice, a handful of interrupts is well within the capability of the system.
- **Power:** GPIO pins source or sink a maximum of 16 mA individually, with a total bank limit. Encoder signal pins draw negligible current when used as inputs with pull-ups; no power budget concern arises.
- **Voltage:** All GPIO inputs must remain within 0 V to 3.3 V. The encoders are passive switches to ground and are inherently safe provided the pull-ups are connected to 3.3 V and not 5 V.

4.6 Setting Up Interrupts in Python

The RPi.GPIO library provides `add_event_detect()` to register a callback function that is invoked when a specified edge transition is detected on a GPIO pin. The callback runs in a separate thread managed by the library, but in a simple synchronous script the main loop can simply read shared variables that the callback updates.

The general pattern for a single interrupt is:

```
import RPi.GPIO as GPIO

GPIO.setmode(GPIO.BCM)

PIN = 17
GPIO.setup(PIN, GPIO.IN, pull_up_down=GPIO.PUD_UP)

def on_edge(channel):
    # called automatically when the edge is detected
    pass

GPIO.add_event_detect(
    PIN,
    GPIO.FALLING,
    callback=on_edge,
```

```
    bouncetime=10        # ignore re-triggers within 10 ms
)
```

4.7 Two Encoders with Multiple Interrupts

For two encoders, four GPIO pins are used (A and B for each encoder). An interrupt is registered on the A pin of each encoder. When the interrupt fires, the handler reads the B pin to determine direction and updates the appropriate software counter.

```
import RPi.GPIO as GPIO
import time

# --- Pin assignments ---
ENC1_A = 17    # Volume encoder
ENC1_B = 27
ENC2_A = 22    # Brightness encoder
ENC2_B = 23

# --- Shared state ---
volume     = 50    # 0-100
brightness = 50    # 0-100

GPIO.setmode(GPIO.BCM)

for pin in [ENC1_A, ENC1_B, ENC2_A, ENC2_B]:
    GPIO.setup(pin, GPIO.IN, pull_up_down=GPIO.PUD_UP)


def on_volume(channel):
    global volume
    direction = 1 if GPIO.input(ENC1_B) else -1
    volume = max(0, min(100, volume + direction))


def on_brightness(channel):
    global brightness
    direction = 1 if GPIO.input(ENC2_B) else -1
    brightness = max(0, min(100, brightness + direction))


GPIO.add_event_detect(ENC1_A, GPIO.FALLING,
                      callback=on_volume,     bouncetime=10)
GPIO.add_event_detect(ENC2_A, GPIO.FALLING,
                      callback=on_brightness, bouncetime=10)
```

```
def main():
    print("Turning encoders. Ctrl-C to exit.")
    try:
        while True:
            # Main loop runs normally; callbacks fire asynchronously
            print(f"Volume: {volume:3d}   Brightness: {brightness:3d}",
                  end="\r")
            time.sleep(0.1)
    except KeyboardInterrupt:
        pass
    finally:
        GPIO.cleanup()


if __name__ == "__main__":
    main()
```

Key points in this implementation:

- The B pin of each encoder is read inside the interrupt callback, not registered as a separate interrupt. Only the A pin triggers the interrupt.
- The `bouncetime=10` argument suppresses re-triggers within 10 ms, providing software debouncing in addition to any hardware RC filtering.
- The `global` keyword allows the callback (running in the RPi.GPIO interrupt thread) to modify the counter variables read by the main loop.
- The counters are clamped to their valid ranges inside each callback.
- `GPIO.cleanup()` is called on exit to release all pin reservations and remove event detection.

4.8 Temporarily Reassigning an Encoder

Because the encoder counters are simple integer variables, reassigning an encoder to a different parameter requires only redirecting which variable the callback modifies. The cleanest approach is to use a function that unregisters the existing callback and registers a new one:

```
def reassign_encoder1(new_variable: list, label: str) -> None:
    """
    Temporarily redirect encoder 1 to control a different parameter.
    new_variable must be a mutable container, e.g. a list of one int.
    """
    GPIO.remove_event_detect(ENC1_A)

    def on_reassigned(channel):
```

```
        direction = 1 if GPIO.input(ENC1_B) else -1
        new_variable[0] = max(0, min(100, new_variable[0] + direction))

    GPIO.add_event_detect(ENC1_A, GPIO.FALLING,
                          callback=on_reassigned, bouncetime=10)
    print(f"Encoder 1 now controls: {label}")
```

To restore the original volume function, call `reassign_encoder1` again with the `volume` container and label.

4.9 Adafruit 1651 Touchscreen Interrupt (TSC2007)

The Adafruit 1651 display uses the **TSC2007** resistive touch controller, which communicates over I2C. The TSC2007 provides a **PENIRQ** pin (active low) that is asserted whenever the screen is touched. Connecting this pin to a GPIO and registering a falling-edge interrupt allows the application to respond to touch events immediately, without polling the I2C bus in a continuous loop.

```
TOUCH_IRQ = 24    # PENIRQ from TSC2007

GPIO.setup(TOUCH_IRQ, GPIO.IN, pull_up_down=GPIO.PUD_UP)

def on_touch(channel):
    """Read touch coordinates from TSC2007 over I2C."""
    # Issue a read command to the TSC2007 over I2C here
    # e.g. using smbus2 or adafruit_tsc2007 library
    pass

GPIO.add_event_detect(TOUCH_IRQ, GPIO.FALLING,
                      callback=on_touch, bouncetime=50)
```

The PENIRQ pin requires a pull-up to 3.3 V; the internal RPi pull-up activated by `GPIO.PUD_UP` is sufficient. The PENIRQ line is open-drain on the TSC2007 and safe to connect directly to the GPIO pin provided the pull-up is to 3.3 V.

4.10 Parts List

Reference	Description	Value / Part	Package
ENC1, ENC2	Rotary encoder, mechanical, no push button	e.g. Alps EC11	Through-hol
R1–R4	Signal pull-up resistors (A and B, each encoder)	10 kΩ	0603
C1–C4	Hardware debounce capacitors (A and B, each encoder)	100 nF ceramic	0603

Table 6: Rotary encoder components